

# MULTI TP9

Siyuan CHEN

Xiru ZHANG

25 septembre 2025

## Table des matières

|                                                                               |          |
|-------------------------------------------------------------------------------|----------|
| <b>C. Application producteur / consommateur</b>                               | <b>2</b> |
| Question C1 . . . . .                                                         | 2        |
| Question C2 . . . . .                                                         | 2        |
| Question C3 . . . . .                                                         | 3        |
| <b>D. Synchronisation par bascule SET/RESET</b>                               | <b>4</b> |
| Question D1 . . . . .                                                         | 4        |
| Question D2 . . . . .                                                         | 4        |
| Question D3 . . . . .                                                         | 4        |
| Question D4 . . . . .                                                         | 4        |
| Question D5 . . . . .                                                         | 4        |
| <b>E. Problèmes de synchronisation liés au réordonnement des instructions</b> | <b>5</b> |
| Question E1 . . . . .                                                         | 5        |
| Question E2 . . . . .                                                         | 6        |
| <b>F. Synchronisation par FIFO logicielle</b>                                 | <b>6</b> |
| Question F1 . . . . .                                                         | 6        |
| Question F2 . . . . .                                                         | 6        |
| Question F3 . . . . .                                                         | 6        |
| Question F4 . . . . .                                                         | 6        |
| Question F5 . . . . .                                                         | 7        |
| Question F6 . . . . .                                                         | 7        |
| Question F7 . . . . .                                                         | 7        |
| Question F8 . . . . .                                                         | 9        |
| Question F9 . . . . .                                                         | 9        |
| <b>G. Application logicielle multi-tâches</b>                                 | <b>9</b> |
| Question G1 . . . . .                                                         | 9        |
| Question G2 . . . . .                                                         | 10       |

## C. Application producteur / consommateur

### Question C1

Sur le terminal TTY[0], à chaque itération de sa boucle principale (jusqu'à NMAX fois, ici 50), la tâche productrice affiche un message indiquant la valeur qu'elle vient de transférer (la valeur actuelle de l'itérateur  $n$ , allant de 0 à 49) ainsi que le temps de pause actuel (défini par PRODUCER\_DELAY, ici 100).

Sur le terminal TTY[1], à chaque itération de sa boucle principale (jusqu'à NMAX fois, ici 50), la tâche consommatrice affiche un message indiquant la valeur qu'elle vient de recevoir (la valeur actuelle de l'itérateur  $n$ , allant de 0 à 49) ainsi que le temps de pause actuel (défini par CONSUMER\_DELAY, ici 100).

### Question C2

```
1 proc0:
2     # initialises interrupt vector entries for PROC[0]
3     la      $26,    _interrupt_vector
4     la      $27,    _isr_dma
5     sw      $27,    0($26)          # _interrupt_vector[0] <=
    _isr_dma
6     la      $27,    _isr_ioc
7     sw      $27,    4($26)          # _interrupt_vector[1] <=
    _isr_ioc
8     la      $27,    _isr_timer
9     sw      $27,    8($26)          # _interrupt_vector[2] <=
    _isr_timer
10    la      $27,    _isr_tty_get
11    sw      $27,    12($26)         # _interrupt_vector[3] <=
    _isr_tty_get
12    #initializes the ICU[0] MASK register
13    la      $26,    seg_icu_base
14    addiu   $26,    $26,    0        # ICU[0]
15    li      $27,    0b0000001111    # IRQ_DMA, IRQ_IOC, IRQ_TIM
    [0] & IRQ_TTY[0]
16    sw      $27,    8($26)
17    # initializes stack pointer for PROC[0]
18    la      $29,    seg_stack_base
19    li      $27,    0x10000          # stack size = 64K
20    addu    $29,    $29,    $27      # $29 <= seg_stack_base +
    64K
21    # initializes SR register for PROC[0]
22    li      $26,    0x0000FF13
23    mtc0    $26,    $12              # SR <= 0x0000FF13
24    # jump to main in user mode: main[0]
25    la      $26,    seg_data_base
26    lw      $26,    0($26)          # read main[0]
27    mtc0    $26,    $14              # write it in EPC register
28    eret
29 proc1:
30    # initialises interrupt vector entries for PROC[1]
31    la      $26,    _interrupt_vector
```

```

32      la      $27,      _isr_timer
33      sw      $27,      16($26)          # _interrupt_vector[4] <=
_isr_timer
34      la      $27,      _isr_tty_get
35      sw      $27,      20($26)          # _interrupt_vector[5] <=
_isr_tty_get
36      #initializes the ICU[1] MASK register
37      la      $26,      seg_icu_base
38      addiu   $26,      $26,      32      # ICU[1]
39      li      $27,      0b0000110000      # IRQ_TIM[1], IRQ_TTY[1]
40      sw      $27,      8($26)
41      # initializes stack pointer for PROC[1]
42      la      $29,      seg_stack_base
43      li      $27,      0x20000          # stack size = 64K
44      addu    $29,      $29,      $27      # $29 <= seg_stack_base +
128K
45      # initializes SR register for PROC[1]
46      li      $26,      0x0000FF13
47      mtc0    $26,      $12              # SR <= 0x0000FF13
48      # jump to main in user mode: main[1]
49      la      $26,      seg_data_base
50      lw      $26,      4($26)          # read main[1]
51      mtc0    $26,      $14              # write it in EPC register
52      eret

```

Listing 1 – MIPS Assembly Code

### Question C3

1. **PRODUCER\_DELAY = 10, CONSUMER\_DELAY = 1000** : Le nombre de cycles CPU pour le consommateur (751,367) est significativement plus élevé que pour le producteur (202,402). Cela montre que le producteur fonctionne beaucoup plus rapidement que le consommateur, qui ne parvient pas à lire les valeurs de manière contiguë. En conséquence, TTY[1] affiche des valeurs non contiguës (ou données périmées) de 2 à 49, tandis que TTY[0] montre une séquence continue de 0 à 49.
2. **PRODUCER\_DELAY = 1000, CONSUMER\_DELAY = 10** : Le nombre de cycles CPU du producteur (754,903) est largement supérieur à celui du consommateur (195,327), indiquant que le producteur est beaucoup plus lent. Le consommateur lit donc plusieurs fois la même valeur avant qu'une nouvelle soit produite. Ainsi, TTY[0] affiche une séquence de 0 à 49, alors que TTY[1] montre les valeurs de 0 à 11, indiquant la lecture répétée de données potentiellement périmées ou répétées.

## D. Synchronisation par bascule SET/RESET

### Question D1

La variable SYNC joue un rôle crucial dans la prévention des accès concurrents incohérents au tampon BUFFER par les tâches producteur et consommateur. Grâce à un mécanisme de synchronisation, tel qu'un sémaphore, un mutex, ou une variable de condition, SYNC assure qu'à tout moment, une seule tâche, soit le producteur, soit le consommateur, puisse effectuer son opération sur le BUFFER.

### Question D2

- Avec PRODUCER\_DELAY = 10 et CONSUMER\_DELAY = 1000, la durée d'exécution est de 749 022 cycles.
- Avec PRODUCER\_DELAY = 1000 et CONSUMER\_DELAY = 10, la durée d'exécution est de 756 059 cycles.
- Avec PRODUCER\_DELAY = 100 et CONSUMER\_DELAY = 100, la durée d'exécution est considérablement réduite à 256 564 cycles.

Ces résultats montrent que l'efficacité du mécanisme de synchronisation varie en fonction du rapport des délais entre le producteur et le consommateur. Des délais asymétriques entraînent une durée d'exécution plus longue, tandis qu'un équilibre entre les délais des deux tâches optimise les performances en minimisant le temps d'exécution global.

### Question D3

Si chaque processeur cache localement les valeurs de SYNC et BUFFER, un processeur peut modifier sa copie locale de l'une de ces variables sans que cette modification soit immédiatement visible pour l'autre processeur.

### Question D4

Lorsque le mécanisme de snoop est désactivé, on observe un blocage immédiat des deux tâches dès la première itération.

### Question D5

Le principal coût matériel causé par le mécanisme de SNOOP, en dehors de l'automate SNOOP\_FSM lui-même, est lié à la logique nécessaire pour supporter les opérations de surveillance et de maintien de la cohérence de cache. Voici des exemples :

- Le mécanisme nécessite une logique complexe pour comparer les adresses surveillées avec celles des données en cache, afin de déterminer si une opération externe affecte les données en cache (hit de cache) ou non.
- Des registres supplémentaires sont nécessaires pour stocker les adresses surveillées, les états des données en cache (par exemple, les bits de validité, les états de cohérence)

## E. Problèmes de synchronisation liés au réordonnancement des instructions

### Question E1

```
1  __attribute ((constructor)) void producer()
2  {
3      int x;
4      int n;
5      int tempo = PRODUCER_DELAY;
6      int val;
7
8      tty_printf("*** Starting task producer on processor %d ***\n\n",
9                  procid());
10
11     for(n = 0 ; n < NMAX ; n++)
12     {
13         for(x = 0 ; x < tempo ; x++) asm volatile ("");
14         while(SYNC!=0);
15         BUF = n;
16         SYNC = 1;
17         tty_printf("transmitted value : %d      temporisation = %d\n",
18                     n , tempo);
19     }
20
21     tty_printf("\n*** Completing producer at cycle %d ***\n",
22                 proctime());
23     exit();
24 } // end producer()
25
26 /*****
27  *****/
28 __attribute ((constructor)) void consumer()
29 {
30     int x;
31     int n;
32     int val;
33     int tempo = CONSUMER_DELAY;
34
35     tty_printf("*** Starting task consumer on processor %d ***\n\n",
36                 procid());
37
38     for(n = 0 ; n < NMAX ; n++)
39     {
40         for(x = 0 ; x < tempo ; x++) asm volatile ("");
41         while(SYNC!=1);
42         val = BUF;
43         SYNC = 0;
44         tty_printf("received value : %d      temporisation = %d\n",
45                     val, tempo);
46     }
```

```

43     tty_printf("\n*** Completing consumer at cycle %d ***\n",
44     proctime());
45     exit();
46 }

```

Listing 2 – Producteur & Consommateur

## Question E2

Dans le segment de texte des deux processus, une instruction supplémentaire ‘sync’ est ajoutée là où le macro est utilisé. Pour chaque transfert, cela ajoute trois cycles supplémentaires.

## F. Synchronisation par FIFO logicielle

### Question F1

Dans un environnement multitâche, si un canal de communication est conçu pour être partagé par plusieurs tâches, il est souvent déclaré comme une variable globale. De cette manière, toutes les tâches qui ont besoin d’accéder à ce canal de communication peuvent facilement y faire référence.

Les différents champs de cette structure sont :

- buf : Tampon mémoire à lire/écrire
- ptr : Numéro de la première case pleine à lire (modulo N)
- ptw : Numéro de la première case libre à écrire (modulo N)
- sts : Nombre total de données présentes dans la FIFO (N + 1 valeurs possibles)
- depth : La taille du tampon mémoire
- lock : La structure de verrouillage utilisée pour synchroniser l’accès au FIFO.

### Question F2

Un simple verrou peut causer le problème de famine car les tâches ne garantissent pas l’équité de l’accès au verrou ; la chance d’acquisition du verrou est aléatoire plutôt qu’un ordre fixé.

### Question F3

Les deux fonctions ont le même argument, un pointeur de type **lock\_t**. Cet argument permet de réaliser une fonction de verrouillage.

### Question F4

**lock\_acquire** : Elle est utilisée pour acquérir un verrou. La fonction commence par utiliser une opération atomique pour incrémenter le membre free de la structure de ver-

rouillage, ce qui retourne le numéro du prochain ticket disponible. Ensuite, elle sonde le membre current de la structure de verrouillage, jusqu'à ce que la valeur de current soit égale au numéro du ticket, à ce moment-là, l'acquisition du verrou est réussie.

La raison pour laquelle la fonction `atomic_increment()` est écrite en assembleur est due à la nécessité d'une exécution atomique.

```
1 00000000 <atomic_increment>:
2   0: 27bdfff0   addiu sp,sp,-16
3   4: afbe000c   sw   s8,12(sp)
4   8: 03a0f025   move  s8,sp
5   c: afc40010   sw   a0,16(s8)
6  10: afc50014   sw   a1,20(s8)
7  14: 8fc20010   lw   v0,16(s8)
8  18: 8fc30014   lw   v1,20(s8)
```

Listing 3 – Segment of `<atomic_increment>`

Dans l'ensemble, ce code définit le pointeur de cadre au début de la fonction et enregistre les paramètres qui peuvent être destinés à des opérations atomiques ultérieures. Cependant, ce code ne montre pas la mise en œuvre de l'opération d'augmentation de l'atomicité(l1 et sc)

## Question F5

**lock\_release** : Elle est utilisée pour libérer un verrou. La fonction effectue d'abord une opération sync pour assurer l'achèvement des opérations en mémoire précédentes, puis elle incrémente la valeur du membre current de la structure de verrouillage de 1, indiquant que la tâche suivante détenant un ticket peut maintenant acquérir le verrou.

La raison pour laquelle `lock_release()` n'a pas besoin d'être écrite en assembleur est que l'opération qu'elle effectue, incrémenter une variable partagée (current), peut être réalisée de manière atomique à l'aide des opérations fournies par de nombreux langages de haut niveau, y compris C.

## Question F6

Si la tâche ne libère pas le verrou lorsque la FIFO n'est pas dans un état opérationnel, elle continuera à occuper le verrou, ce qui entraînera le système dans un état de blocage où aucune donnée ne peut être transférée.(deadlock)

## Question F7

```
1 void fifo_write(fifo_t* fifo, int* val)
2 {
3     int done = 0;
4
5     while(done == 0)
6     {
```

```

7      lock_acquire( (lock_t*)&fifo->lock) );
8
9      if(fifo->sts == fifo->depth)
10     {
11         lock_release( (lock_t*)&fifo->lock) );
12     }
13     else
14     {
15         fifo->buf[fifo->ptw] = *val;
16         fifo->ptw = (fifo->ptw++) % fifo->depth;
17         fifo->sts++;
18         lock_release( (lock_t*)&fifo->lock) );
19         done = 1;
20     }
21 }
22 }

```

Listing 4 – fifo\_write

```

1 void fifo_read(fifo_t* fifo, int* val)
2 {
3     int done = 0;
4
5     while(done == 0)
6     {
7         lock_acquire( (lock_t*)&fifo->lock) );
8
9         if(fifo->sts == 0)
10        {
11            lock_release( (lock_t*)&fifo->lock) );
12        }
13        else
14        {
15            *val = fifo->buf[fifo->ptr];
16            fifo->ptr = (fifo->ptr++) % fifo->depth;
17            fifo->sts--;
18            lock_release( (lock_t*)&fifo->lock) );
19            done = 1;
20        }
21    }
22 }

```

Listing 5 – fifo\_read

## Question F8

| DEPTH    | 1      | 2      | 4      | 8      |
|----------|--------|--------|--------|--------|
| Producer | 512434 | 491598 | 517293 | 466139 |
| Consumer | 513425 | 518487 | 555326 | 479870 |

TABLE 1 – Le coût d’une opération de communication entre deux tâches

Avec l’augmentation de la profondeur du tampon, le nombre de cycles nécessaires pour le producteur montre une tendance générale à la baisse, ce qui suggère qu’un tampon plus grand peut améliorer l’efficacité du producteur. Le consommateur présente le plus grand nombre de cycles à une profondeur de 4, suivi d’une diminution à une profondeur de 8, indiquant qu’un tampon trop grand ou trop petit n’est pas optimal pour l’efficacité du consommateur.

## Question F9

| DEPTH    | 1      | 2      | 4      | 8      |
|----------|--------|--------|--------|--------|
| Producer | 319364 | 318358 | 287375 | 311485 |
| Consumer | 319471 | 318465 | 287484 | 311484 |

TABLE 2 – Le coût moyen d’une opération de communication entre deux tâches

## G. Application logicielle multi-tâches

### Quesiton G1

Dans le cadre de la configuration du fichier `reset.s` pour lancer six tâches sur six processeurs différents, il est uniquement nécessaire d’apporter des modifications au niveau du point où chaque processeur effectue le saut vers le mode utilisateur :

```
# Pour chaque processeur, ajuster la séquence de saut :
la    $26,    seg_data_base
lw    $26,    OFFSET($26)    # OFFSET est ajusté pour chaque processeur
mtc0  $26,    $14
eret
```

**Valeur d’OFFSET spécifique à chaque processeur :**

- **proc0** : OFFSET = 0 (tâche productrice)
- **proc1** : OFFSET = 4 (tâche consommatrice)
- **proc2 à proc5** : OFFSET = 8 (tâches de traitement)

## **Question G2**

En définissant des verrous indépendants pour chaque canal, les tâches peuvent accéder simultanément à différents canaux sans se bloquer mutuellement. Le parallélisme inhérent à l'application peut être exploité plus efficacement, permettant à plusieurs opérations de communication (canaux de lecture ou d'écriture) de se produire en parallèle.